The Anatomy of an Agent: Components and Core Logic

Explore the key parts of an agentic RAG system by understanding its core components: the brain (reasoning engine), tools (capabilities), planner (decision maker), and memory (context and state). This lesson guides you through how these elements work together to enable multi-step reasoning, tool use, and dynamic problem solving beyond standard retrieval pipelines.

In our last lesson, we established the “why” behind agentic RAG. We saw that while standard RAG is powerful, its rigid, linear pipeline struggles with complex, multi-step problems or any task that requires external tools. We concluded that to overcome these limits, we need a more dynamic system: one that can reason, plan, and act.

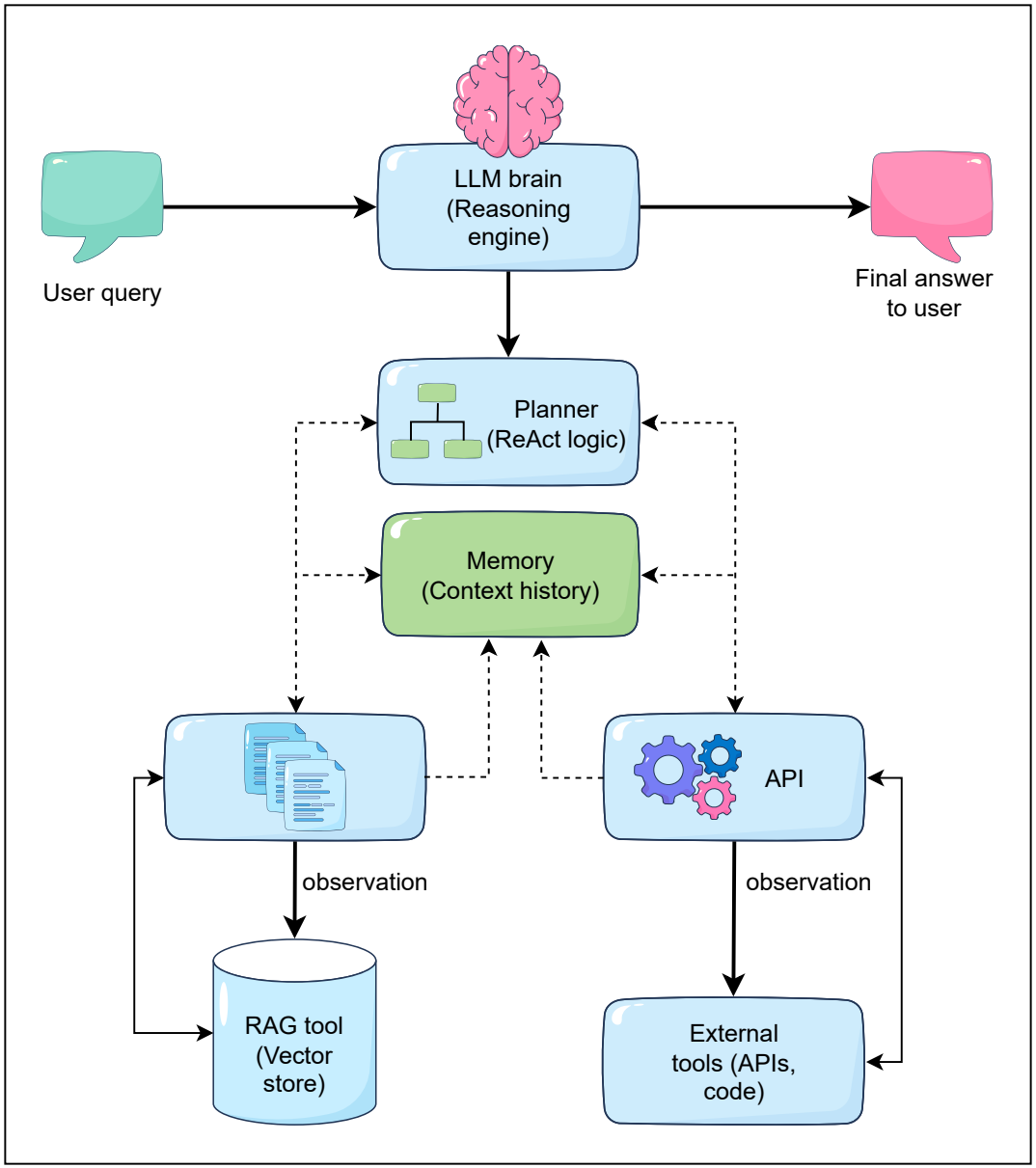
In this lesson, we’ll deconstruct exactly what such a system looks like. We’ll transition from the high-level concept of an “agent” to a concrete understanding of how its internal components work together to create a truly intelligent and responsive RAG system.

What makes a system “agentic”?

To start, let’s remember the core problem with a standard RAG pipeline: it’s a fixed process. It always follo same steps: retrieve, augment, and generate. This makes it great for direct questions, but poorly suited for

Ask

anything more complex. An agent, on the other hand, is not a fixed pipeline; it’s a dynamic problem-solver that uses RAG as a foundational capability.



Ask

An architectural overview of an agentic RAG system

To make this concept intuitive, we'll use an analogy: think of an intelligent agent as a film director . A director's success depends on a complete system working in harmony.

- **The brain:** This is the director's creative vision, knowledge of cinema, and experience (the LLM).
- **The tools:** The director is powerless without their cameras, crew, actors, and equipment (the agent's tools). The most important of these is the script library that they can reference (the RAG pipeline).
- **The plan:** This is the shooting script and storyboard that the director decides to follow, adapting it as scenes are filmed (the planner/agentic loop).
- **The memory:** This is the director's ability to remember which take was the best and how the current scene connects to the previous one (the agent's memory).

In code, this conceptual loop can be imagined as:

```
while not goal_reached:
    thought = llm.think(current_state)
    action = planner.choose_tool(thought)
    observation = action.execute()
    memory.update(thought, action, observation)
```

This simple pseudo-code shows how the agent continuously thinks, acts, and observes until the task is complete

The key insight here is that a great director isn't just a brain with a vision; they are a complete system tha tools and follows a plan to create something new . Likewise, an agent isn't just an LLM; it's a complete system that uses tools to execute a plan.

Ask

Now, that we have a high-level analogy, let's break down the agent into its four essential, functional components. Understanding the role of each part is the key to designing and debugging effective agents.

Component 1: The brain (the reasoning engine)

The brain is the core processing unit of the agent. While it's not the entire agent, it's the part that "thinks." In our case, this is a large language model (LLM). In an agentic RAG context, its job is to act as a central coordinator with specific RAG-related tasks.

- **Understanding intent:** Interpreting what the user is asking for.
- **Decomposition:** Breaking down a large problem into steps, determining when a RAG retrieval is necessary.
- **Tool selection:** Looking at the available tools and deciding if the RAG pipeline is sufficient or if another tool is needed.
- **Synthesis:** Combining the results from its tools, especially the retrieved context from the RAG pipeline, into a final, coherent, and grounded answer for the user.



Ask

Note on model selection:

For our course, we are using the Groq API with Llama 3 as our agent's brain. We've chosen this stack

> because it's free to use and its speed is crucial for making the agent's reasoning process feel interactive, and responsive.
>
> Groq also supports `llama-4-scout-17b-16e-instruct`, which is a newer, mixture-of-experts (MoE) model with a longer context window (up to 131K tokens) and improved factual grounding. You're welcome to experiment with it by changing your model name in the code to `MODEL = "llama-4-scout-17b-16e-instruct"`. However, for this course, we'll continue using Llama 3 as our default. It offers more stable, reproducible outputs and aligns well with Groq's ultra-low-latency performance, which is ideal for teaching and debugging the agentic reasoning loop.

Without a powerful reasoning engine, a RAG system can only answer the one specific question it was asked. It cannot decide to perform a RAG query, then use a calculator on the result, and then perform a second RAG query. The LLM brain enables these multi-step, intelligent workflows.

Component 2: The tools (the capabilities)

Tools are what give an agent its power. They are the components that allow an agent to interact with the world beyond the static knowledge in the LLM's training data. An agent without tools is just a chatbot. In our analogy, the director is powerless without their cameras, crew, and actors; these are the tools that bring their vision to life.

Ask

The key idea is that we expose our data sources and functions to the agent as “tools” that it can choose to call when it needs to. In an agentic RAG system, the RAG pipeline is the primary, indispensable tool; it’s the script library that the director constantly references. Other tools are supplementary, like special effects or a research department, enhancing the core story.

Throughout this course, we will build agents that use several types of tools, as categorized below:

Tool Category	Description	Primary RAG Context	Supplementary Use Case
Data Retrieval	Accessing information from a knowledge base.	RAG pipeline for local documents.	Querying a structured SQL database.
Live Information	Fetching real-time, external data.	API callers to enrich retrieved RAG context.	Getting live weather forecasts.
Computation	Performing calculations or executing code.	Code interpreters to analyze RAG results.	Calculating financial models.

Internally, when the agent decides to use a tool, it’s essentially making a function call . We can conceptualize a tool’s structure in code like this:

```
# A conceptual look at how a "Tool" is structured
class ArxivSearchTool:
    name = "arxiv_search"
    description = "Use this to search for scientific papers on arXiv."

    def __call__(self, query: str):
```

Ask

```
    def __call__(self, query: str):
        # ... API call logic would go here ...
        print(f"Searching arXiv for: {query}")
        # ... returns results ...
```

A conceptual Python class representing an agent's tool. The name and description are critical for the agent's planner (explained below) to understand what the tool does and when to use it

Tools transform RAG from a simple lookup mechanism into a dynamic component of a larger system. An agent can use the RAG tool to retrieve factual data. It can then feed that data into another tool (like a calculator or API caller) for further processing.

Component 3: The planner (the decision maker)

The planner is the underlying logic or prompt that orchestrates the entire agentic process. Think of it as the agent's "operating system"; it's what tells the agent *how* to think and make decisions. For our film director, this is the shooting script and storyboard. It provides the structure and sequence for the entire production.

Its job is to take the user's query and the list of available tools, and then manage the step-by-step reason to arrive at a solution. This process, outlined below, is often called the **core agentic loop**.

Ask

- Analyze the user's goal.

- Choose the best tool to make progress toward that goal.
- Execute the tool.
- Observe the result.
- Repeat this cycle until the goal is complete.

> The ReAct framework, which we will study in depth in the next lesson, is a specific and powerful implementation of this planning component.

The planner elevates RAG from a reactive system to a proactive one. It can decide that a RAG query is too broad, refine the query, and try again, or decide that the answer from RAG needs to be enriched with another data source. It's the component that intelligently decides when and how to consult the RAG "script library."

1. Wait, aren't the 'brain' and the 'planner' overlapping? They both seem to handle similar types of things.

Show Answer

Did you find this helpful?

Component 4: The memory (context and state)

Ask

For an agent to solve any non-trivial, multi-step task, it must be able to remember what it has already done and learned. Memory provides context and statefulness to the agent. There are two primary types of memory an

agent uses.

- **Short-term memory (the scratchpad):** A director's immediate memory of which take was the best one and how the current scene connects to the previous one.
    - **What it stores:** This stores the immediate context of the current task. It includes the history of the conversation and, crucially, the thought -> act -> observe steps that the agent has already taken.
    - **Purpose:** This is how an agent remembers what it did in step 2 when it gets to step 3 of a plan. In a RAG context, this is where the agent "holds" the retrieved documents, so it can reason about them in the next step.
- **Long-term memory (the knowledge base):** A director's entire library of films and scripts that they can reference for inspiration.
    - **What it stores:** This is where an agent can store and retrieve information across many different conversations or tasks.
    - **Purpose:** For any agentic RAG system, the RAG vector store is the agent's primary long-term memory. For a simple prototype, this might be an in-memory index. However, for production systems, this is typically a dedicated vector database like ChromaDB, Weaviate, or Pinecone. It is the externalized knowledge that the agent can access on demand via its RAG tool.

Ask

Memory is the thread that connects the agentic workflow. Short-term memory allows the agent to use the output of a RAG query as the input for another tool. Long-term memory (the vector store) is the foundation of the RAG system itself.

Test your understanding

You built an AI research assistant using agentic RAG. It retrieves papers correctly but forgets which documents it already summarized, re-fetching the same chunks repeatedly.

- **Q1:** Which component is likely misconfigured, brain, tools, planner, or memory?
- **Q2:** Why do you think so?
- **Q3:** How would you fix it?

Detail your solution and reasoning.

Evaluate your answer!

H1 H2 H3 | B I | | | | |

Write your answer here...

Ask

Conclusion: A complete system

In this lesson, we deconstructed the AI agent and moved from a vague concept to a concrete architectural understanding. We learned that an agent is not just an LLM; it's a complete system where multiple components must work in harmony to make the RAG process more intelligent and dynamic.

Using our film director analogy, we can now see the full picture. The brain (LLM) provides the reasoning and vision, the tools provide the ability to act, the memory provides context and continuity, and the planner orchestrates the entire production. Without any one of these pieces, the system's effectiveness diminishes significantly.

To consolidate everything we've learned, here is a final summary table:

Component	Film Director Analogy	Core Function in Agentic RAG	Consequence If Missing
Brain (LLM)	Director's Vision	Reasons over retrieved context and plans the next action.	Inability to handle complex tasks.
Tools	Cameras and Crew	The RAG pipeline is the primary tool for accessing knowledge.	A "brain in a jar"; no real-world interaction.
Planner	Shooting Script	Decides when and how to use the RAG tool vs. other tools.	Chaotic and inefficient actions.
Memory	Continuity Notes	The vector store is the long-term memory of the system.	Inability to perform multi-step tasks.

Ask

We’ve focused heavily on the “planner” as the agent’s core decision-maker, and we defined its job as managing the step-by-step reasoning loop. The ReAct framework is a specific and powerful implementation of this component

Ask